Trabajo práctico número 3

Pilas

1) Se desea codificar una pila dinámica. Los nodos contienen números enteros. Cargar la pila con 10 nodos, recorrerla y mostrar su contenido por pantalla.

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	int dato;
	struct nodo *sig;
};

int main()
{
	struct nodo *p, *primero;
	int n;

	p=NULL;
	primero=NULL;

	printf("Ingresar numeros eteros (0 para terminar de ingresar) \n");
	printf("Ingrese un entero: ");
	scanf("%d", &n);

	while( n!=0 )			//Ingreso datos
	{
		p=(struct nodo*)malloc(sizeof(*p));
		if( p==NULL )
		{
			printf("No se pudo gestionar la memoria \n");
			return 1;
		}
		else
		{
			(*p).dato=n;
			(*p).sig=primero;
			primero=p;

			printf("Ingrese un entero: ");
			scanf("%d", &n);
		}
	}


	while( p!=NULL)				//Muestro por pantalla
	{
		printf("%d \n", (*p).dato );
			p=(*p).sig;
	}

	printf("\n\n\n");

	system("PAUSE");
	return 0;
}
```

2) Codificar un programa que permita cargar una pila dinámica con N caracteres ingresados desde el teclado. Recorrerla y mostrar su contenido por pantalla.

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	char dato;
	struct nodo *sig;
};

int main()
{
	struct nodo *p, *base;
	char c;

	p=NULL;
	base=NULL;

	printf("Ingrese caracteres (z para terminar) \n");
	printf("Ingrese un caracter: ");

	while( (c=getchar()) != 'z' )
	{
		p=(struct nodo*)malloc(sizeof(*p));

		if(p==NULL)
		{
			printf("No se pudo gestionar la memoria \n");
			return 1;
		}
		else
		{
			(*p).dato=c;
			(*p).sig=base;
			base=p;

			printf("Ingrese un caracter: ");
		}
	}

	printf("\nLa lista ingresada es la siguiente \n\n");

	while(p!=NULL)
	{
		printf("%c \n", (*p).dato);
		p=(*p).sig;
	}

	system("PAUSE");
	return 0;
}
```

3) Se desea codificar una pila dinámica. Los nodos contienen Apellido y edad de personas. Cargar la pila hasta que se ingrese la edad 999. Se desea recorrerla y mostrar por pantalla la información de los nodos que representen personas mayores de edad (21 años).

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	char apellido[30];
	int edad;
	struct nodo *sig;
};

int getint(void);

int main()
{
	struct nodo *p, *base;
	int n;

	base=NULL;
	p=NULL;

	printf("Ingrese nombre y apellido de las personas (edad 999 para terminar) \n");


	printf("\n----------------------------------------\n");
	printf("\nEdad de la persona: ");

	while( (n=getint())!=999 )
	{
		p=(struct nodo*)malloc(sizeof(*p));
		if(p==NULL)
		{
			printf("No se pudo gestionar la memoria \n");
			return 1;
		}
		else
		{
			printf("Apellido de la persona: ");
			scanf("%s", (*p).apellido);

			(*p).edad=n;
			(*p).sig=base;
			base=p;

			printf("\n----------------------------------------\n");
			printf("\nEdad de la persona: ");
		}
	}
	printf("\nLa lista de los mayores de edad es: \n");

	while(p!=NULL)
	{
		if( (*p).edad>=21 )
		{
			printf("Apellido: %s   Edad: %d \n", (*p).apellido, (*p).edad);
		}
		p=(*p).sig;
	}
```

```
	system("PAUSE");
	return 0;
}

int getint(void)
{
	int n;
		scanf("%d", &n);
	return n;
}
```

4) Se desea codificar una pila dinámica. Los nodos contienen Apellido y edad de la persona. Cargar la pila hasta que se ingrese la edad 999. Se desea recorrerla y generar otra que solo contenga los nodos pertenecientes a personas mayores de edad (21 años). Mostrar por pantalla.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct nodo
{
	char apellido[30];
	int edad;
	struct nodo *sig;
};

int getint(void);

int main()
{
	struct nodo *p, *base, *q, *base2;
	int n;

	base=NULL;	base2=NULL;
	p=NULL;		q=NULL;

	printf("Ingrese nombre y apellido de las personas (edad 999 para terminar) \n");
	printf("\n----------------------------------------\n");
	printf("\nEdad de la persona: ");

	while( (n=getint())!=999 )
	{
		p=(struct nodo*)malloc(sizeof(*p));
		if(p==NULL)
		{
			printf("No se pudo gestionar la memoria \n");
			return 1;
		}
		else
		{
			printf("Apellido de la persona: ");
			scanf("%s", (*p).apellido);
			(*p).edad=n;
			(*p).sig=base;
			base=p;
			printf("\n----------------------------------------\n");
			printf("\nEdad de la persona: ");
		}
	}
```

```
	while(p!=NULL)                                  //Algoritmo para copiar mayores de 21
	{                                               //en una nueva pila
		if( (*p).edad>=21 )
		{
			q=(struct nodo*)malloc(sizeof(*q));

			if(q==NULL)
			{
				printf("No se pudo gestionar la memoria\n");
				return 1;
			}
			else
			{
				(*q).edad=(*p).edad;
				strcpy( (*q).apellido, (*p).apellido );    //copia cadenas
				(*q).sig=base2;
				base2=q;
			}
		}
		p=(*p).sig;
	}

	printf("\nLa lista de los mayores de edad es: \n");

	while(q!=NULL)		//imprimo lo que hay en la nueva pila
	{
			printf("Apellido: %s   Edad: %d \n", (*q).apellido, (*q).edad);
		q=(*q).sig;
	}

	system("PAUSE");
	return 0;
}

int getint(void)
{
	int n;
		scanf("%d", &n);
	return n;
}
```

5) Se desea codificar una pila dinámica. Los nodos contienen números enteros. Se deberá cargarla hasta que el usuario decida dejar de hacerlo, mediante ingreso por teclado. Recorrerla y mostrar por pantalla la mitad inferior del contenido de la estructura.

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	int dato;
	struct nodo *sig;
};

int getint(void);

int main()
{
	struct nodo *p, *base;
	int n, i, tam=0;
```

```
	p=NULL;
	base=NULL;

	printf("Ingrese datos (0 para terminar)\n");

	printf("Ingrese dato: ");
	while( (n=getint()) != 0)
	{
			p=(struct nodo*)malloc(sizeof(*p));
			if(p!=NULL)
			{
				(*p).dato=n;
				(*p).sig=base;
				base=p;
				printf("Ingrese dato: ");
			}
			else
			{
				printf("No hay memoria\n");
				return 1;
			}
	}

	//cuento nodos
	while(p!=NULL)
	{
		tam++;
		p=(*p).sig;
	}

	p=base;
	for(i=0 ;i<tam/2 ;i++)  //me situo en la mitad de la pila
	{
		p=(*p).sig;
	}

	while(p!=NULL)    //imprimo los nodos inferiores a la mitad
	{
		printf("%d  \n", (*p).dato);
		p=(*p).sig;
	}


	system("PAUSE");
	return 0;
}

int getint(void)
{
	int n;
		scanf("%d", &n);
	return n;
}
```

6) Leer y almacenar en un arreglo estático los nombres de N alumnos, junto con su promedio general y un entero que represente el número de inasistencias a clase.
Generar una pila dinámica con la información del arreglo. Recorrerla y mostrar mediante un menú, las siguientes opciones:

a) la información del nodo que contiene el promedio mayor

b) la información del nodo que tiene menor número de inasistencias.

c) Los nombres de los alumnos cuyo promedio sea mayor a 7.00.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define tam 3

struct nodo
{
	char nombre[30];
	char apellido[30];
	int faltas;
	float promedio;
	struct nodo *sig;
};

struct alumno
{
	char nombre[30];
	char apellido[30];
	int faltas;
	float promedio;
}ficha[tam];

int main()
{
	struct nodo *p, *base, *q, *base2;
	int  opcion, i, min;
	float max;

	do
	{
		printf("\n\n----MENU----\n");
		printf("1. Ingresar datos \n");
		printf("2. Alumnos con mayor promedio\n");
		printf("3. Alumnos con menos inasistencias \n");
		printf("4. Alumnos con promedio mayor a 7 \n");
		printf("5. Salir \n");
		printf("Ingrese la opcion elegida: ");
		scanf("%d", &opcion);

		switch(opcion)
		{
			case 1:
				printf("\n------------Ingreso de datos------------\n");
				for(i=0 ;i<tam ;i++)
				{
					printf("\n---- ALUMNO %d ----\n", i+1);
					printf("Nombre: ");
					scanf("%s",  ficha[i].nombre);
					printf("Apellido: ");
					scanf("%s", ficha[i].apellido);
					printf("Inasistencias: ");
```

```
                scanf("%d", &ficha[i].faltas);
                printf("Promedio: ");
                scanf("%f", &ficha[i].promedio);
        }

        p=NULL;
        base=NULL;
                                //Copia elementos del arreglo en la pila
        for(i=0 ;i<tam ;i++)
        {
                p=(struct nodo*)malloc(sizeof(*p));
                if(p==NULL)
                {
                        printf("No se pudo gestionar la memoria\n");
                        return 1;
                }
                else
                {
                        strcpy((*p).nombre, &ficha[i].nombre[0]);
                        strcpy((*p).apellido, &ficha[i].apellido[0]);
                        (*p).faltas=ficha[i].faltas;
                        (*p).promedio=ficha[i].promedio;
                        (*p).sig=base;
                        base=p;
                }
        }
    break;
case 2:
    p=base;
    max=0;
    base2=NULL;
        while(p!=NULL)
        {
                if( (*p).promedio>=max )
                {
                        q=(struct nodo*)malloc(sizeof(*q));
                        if(q==NULL)
                        {
                                printf("No se pudo gestionar la memoria \n");
                                return 1;
                        }
                        else
                        {
                                max=(*p).promedio;
                                (*q).promedio=(*p).promedio;
                                strcpy( (*q).nombre, (*p).nombre );
                                strcpy( (*q).apellido, (*p).apellido );
                                (*q).sig=base2;
                                base2=q;
                        }
                }
                p=(*p).sig;
        }
        printf("\nEl mayor promedio es: %.2f: \n", max);
        while(q!=NULL)
        {
                printf("Del alumno %s %s\n", (*q).nombre, (*q).apellido);
                q=(*q).sig;
        }
    free(q);
    break;
```

```
case 3:
        p=base;
        min=999;
        base2=NULL;
                while(p!=NULL)
                {
                        if( (*p).faltas<=min )
                        {
                                q=(struct nodo*)malloc(sizeof(*q));
                                if(q==NULL)
                                {
                                        printf("No se pudo gestionar la memoria \n");
                                        return 1;
                                }
                                else
                                {
                                        min=(*p).faltas;
                                        (*q).faltas=(*p).faltas;
                                        strcpy( (*q).nombre, (*p).nombre );
                                        strcpy( (*q).apellido, (*p).apellido );
                                        (*q).sig=base2;
                                        base2=q;
                                }
                        }
                        p=(*p).sig;
                }
                printf("\nLos alunos con menos inasistencias son: \n");
                while(q!=NULL)
                {
                        printf("%s %s\n", (*q).nombre, (*q).apellido);
                        q=(*q).sig;
                }
                printf("Con %d inasistencias\n", min);
        free(q);
        break;
  case 4:
        p=base;
        max=7;
        base2=NULL;
                while(p!=NULL)
                {
                        if( (*p).promedio>=max )
                        {
                                q=(struct nodo*)malloc(sizeof(*q));
                                if(q==NULL)
                                {
                                        printf("No se pudo gestionar la memoria \n");
                                        return 1;
                                }
                                else
                                {
                                        max=(*p).promedio;
                                        (*q).promedio=(*p).promedio;
                                        strcpy( (*q).nombre, (*p).nombre );
                                        strcpy( (*q).apellido, (*p).apellido );
                                        (*q).sig=base2;
                                        base2=q;
                                }
                        }
                        p=(*p).sig;
                }
                printf("\nLos alumnos con promedio mayor a 7 son: \n");
```

```
                                while(q!=NULL)
                                {
                                        printf("%s %s ", (*q).nombre, (*q).apellido);
                                        printf("\tPromedio: %.2f\n", (*q).promedio);
                                        q=(*q).sig;
                                }
                        free(q);
                        break;
                case 5:
                                printf("\n\n----FIN DEL PROGRAMA----\n\n");
                        break;
                default:
                                system("CLS");
                        printf("Ingrese una opcion valida\n\n");
                        break;
                }
        }while(opcion!=5);
        free(p);

        system("PAUSE");
        return 0;
}
```

7) Generar una estructura dinámica tipo pila que permita guardar en un nodo la información correspondiente a temperatura y presión de una máquina. Generar un menú con las opciones:

- ***a) agregar un nodo nuevo.***
- ***b) borrar un nodo***
- ***c) recorrerla y mostrar su contenido en pantalla.***
- ***d) mostrar por pantalla la máxima y la mínima temperatura, con la posición que ocupan en la estructura***
- ***e) mostrar por pantalla la máxima y la mínima presión, con la posición que ocupan en la estructura***

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
        float temperatura;
        float presion;
        struct nodo *sig;
};

int main()
{
        struct nodo *p, *base;
        float max, min;
        int opcion, i, j, pos1, pos2;

        p=NULL;
        base=NULL;

        do
        {
                printf("----- MENU -----\n");
                printf("1. Agregar nuevo nodo\n");
                printf("2. Borrar nodo\n");
                printf("3. Mostrar contenido en pantalla\n");
                printf("4. Maxima y minima temperatura\n");
                printf("5. Maxima y minima presion\n");
                printf("6. Salir\n");
```

```
printf("\nIngrese la opcion elegida: ");
scanf("%d", &opcion);

switch(opcion)
{
	case 1:
		p=(struct nodo*)malloc(sizeof(*p));
			if(p!=NULL)
			{
				printf("------ Datos -----\n");
				printf("Ingrese temperatura: ");
				scanf("%f", &(*p).temperatura);
				printf("Ingrese presion: ");
				scanf("%f", &(*p).presion);
				(*p).sig=base;
				base=p;
				system("CLS");
			}
			else
			{
				printf("No hay memoria\n");
				return 1;
			}
		break;
	case 2:
			base=(*p).sig;
			free(p);
			system("CLS");
		break;
	case 3:
			while(p!=NULL)
			{
				printf("\n----- Lectura -----\n");
				printf("Temperatura: %.2f\n", (*p).temperatura);
				printf("Presion: %.2f\n", (*p).presion);
				p=(*p).sig;
			}
			p=base;
			printf("\n\n");
		break;
	case 4:
		max=(*p).temperatura-1;
		min=(*p).temperatura+1;
		i=1;
		j=1;
			while(p!=NULL)
			{
				if( (*p).temperatura>max )
				{
					pos1=i;
					max=(*p).temperatura;
				}
			 p=(*p).sig;
			 i++;
			}
		 p=base;
		 while(p!=NULL)
			{
				if( (*p).temperatura<min )
				{

					pos2=j;
```

```
                                        min=(*p).temperatura;
                                }
                                p=(*p).sig;
                                j++;
                                }
                        p=base;
                                system("CLS");
                         printf("La temperatura maxima es: %.2f  ",max );
                         printf("de la lectura %d\n\n", i-pos1);
                         printf("La temperatura minima es: %.2f  ",min );
                         printf("de la lectura %d\n\n", j-pos2);
                        break;
                case 5:
                        max=(*p).presion-1;
                        min=(*p).presion+1;
                        i=1;
                        j=1;
                        p=base;
                                while(p!=NULL)
                                {
                                        if( (*p).presion>max )
                                        {
                                                pos1=i;
                                                max=(*p).presion;
                                        }
                                 p=(*p).sig;
                                 i++;
                                }
                         p=base;
                         while(p!=NULL)
                                {
                                        if( (*p).presion<min )
                                        {
                                                pos2=j;
                                                min=(*p).presion;
                                        }
                                 p=(*p).sig;
                                 j++;
                                }
                                system("CLS");
                         printf("La presion maxima es: %.2f  ",max );
                         printf("de la lectura %d\n\n", i-pos1);
                         printf("La presion minima es: %.2f  ",min );
                         printf("de la lectura %d\n\n", j-pos2);
                        break;
                case 6:
                        system("CLS");
                        printf("Fin del programa\n\n");
                        free(p);
                        break;
                default:
                        system("CLS");
                        printf(" ERROR!! Ingrese una opcion valida\n\n");
                        break;
                }
        }while(opcion!=6);

        system("PAUSE");
        return 0;
}
```

8) Se desea codificar una pila dinámica con métodos para insertar nodo, borrar nodo y recorrer la estructura para mostrar su contenido por pantalla. Los nodos contienen números reales de simple precisión. El programa se termina cuando se ingrese por teclado "s" o "S".
Se deberá crear un menú para solicitar por teclado cual es la tarea que se desea realizar.

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
	float dato;
	struct nodo *sig;
};

int menu(void);

void imprimir(struct nodo *);

struct nodo* pop(struct nodo*);

struct nodo* put(struct nodo*, float);

int main()
{
	struct nodo *p, *base;
	float n;
	int opcion;

	base=NULL;
	p=NULL;

	do
	{
		system("CLS");
		opcion=menu();
		switch(opcion)
		{
			case 1:
				printf("\n----Nuevo Nodo----\n");
				printf("Ingrese dato: ");
				scanf("%f", &n);
				base=put(base, n);
				break;
			case 2:
				base=pop(base);
				break;
			case 3:
				system("CLS");
				p=base;
					printf("La estructura ingresada es: \n");
					imprimir(p);
				p=base;
				break;
			case 4:
					system("CLS");
					printf("Fin del programa\n\n");
				break;
			default:
				system("CLS");
					printf("Ingrese una opcion valida\n\n");
				break;
```

```
                }
        }while(opcion!=4);

        system("PAUSE");
        return 0;
}

int menu(void)
{
        int opc;
                printf("---- MENU ----\n");
                printf("1. Insertar nodo\n");
                printf("2. Borrar nodo \n");
                printf("3. Mostrar por pantalla\n");
                printf("4. Finalizar el programa\n");
                printf("Ingrese opcion elegida: ");
                scanf("%d", &opc);
        return opc;
}

struct nodo* put(struct nodo *p, float n)
{
        struct nodo *q;

                q=(struct nodo*)malloc(sizeof(*q));
                if(q!=NULL)
                {
                        (*q).dato=n;
                        (*q).sig=p;
                        return q;
                }
                else
                {
                        printf("No hay memoria\n");
                }
}

struct nodo* pop(struct nodo *p)
{
        struct nodo *q;

        if(p==NULL)
        {
                printf("La estructtura esta vacia\n");
        }
        else
        {
                q=(*p).sig;
                free(p);
                return q;
        }
}

void imprimir(struct nodo *p)
{
        while(p!=NULL)
        {
                printf("%.2f\n", (*p).dato);
                p=(*p).sig;
        }
}
```

9) Se crean dos pilas, una de ellas con 10 lecturas de concentración de gas provenientes de un sensor y la otra con 10 lecturas del nivel líquido de gas del cartucho que lo contienen (que se corresponden una a una con las lecturas del sensor de concentración). Se desea tener en una sola estructura de tipo pila con ambas informaciones. Mostrar por pantalla las 3 pilas.

```
#include <stdio.h>
#include <stdlib.h>
#define tam 3

struct nodo1
{
	float concentracion;
	struct nodo1 *sig;
};

struct nodo2
{
	float nivel;
	struct nodo2 *sig;
};

struct nodo3
{
	float concentracion;
	float nivel;
	struct nodo3 *sig;
};

int main()
{
	struct nodo1 *p1, *base1;
	struct nodo2 *p2, *base2;
	struct nodo3 *p3, *base3;
	int  i;

	base1=NULL;
	base2=NULL;
	base3=NULL;

	for(i=0 ;i<tam ;i++)
	{
		printf("\n----- Lectura Nro: %d -----\n", i+1);
		p1=(struct nodo1*)malloc(sizeof(*p1));
		if(p1==NULL)
		{
			printf("No hay memoria\n");
			return 1;
		}
		else
		{
			printf("Ingrese la concentrecion de gas: ");
			scanf("%f", &(*p1).concentracion);
			(*p1).sig=base1;
			base1=p1;
		}
		p2=(struct nodo2*)malloc(sizeof(*p2));
		if(p2==NULL)
		{
			printf("No hay memoria\n");
			return 1;
		}
```

```
		else
		{
			printf("Ingrese el nivel de gas en el cartucho: ");
			scanf("%f", &(*p2).nivel);
			(*p2).sig=base2;
			base2=p2;
		}
	}

	while( (p1!=NULL) && (p2!=NULL) )
	{
		p3=(struct nodo3*)malloc(sizeof(*p3));
		if(p3==NULL)
		{
			printf("No hay memoria\n");
			return 1;
		}
		else
		{
			(*p3).concentracion=(*p1).concentracion;
			(*p3).nivel=(*p2).nivel;

			(*p3).sig=base3;
			base3=p3;

			p1=(*p1).sig;
			p2=(*p2).sig;
		}
	}

	p1=base1;
	p2=base2;
	system("CLS");

	printf("Lista de lecturas de concentracion de gas:\n");
	while(p1!=NULL)
	{
		printf("%.2f \t", (*p1).concentracion);
		p1=(*p1).sig;
	}

	printf("\n\nLista de lecturas de nivel de liquido:\n");
	while(p2!=NULL)
	{
		printf("%.2f \t", (*p2).nivel);
		p2=(*p2).sig;
	}

	printf("\n\nLista de lecturas juntas: \n");
	while(p3!=NULL)
	{
		printf("Concentracion: %.2f     Nivel:%.2f\n",(*p3).concentracion, (*p3).nivel);
		p3=(*p3).sig;
	}


	system("PAUSE");
	return 0;
}
```

Colas

1) Se desea codificar una cola dinámica. Los nodos contienen números enteros. Cargar la pila con 10 nodos, recorrerla y mostrar su contenido por pantalla.

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
	int dato;
	struct nodo *sig;
};

int getint(void);
void imprime(struct nodo *);

int main()
{
	struct nodo *p, *q, *base;
	int n;

	p=NULL;		q=NULL;		base=NULL;

	p=(struct nodo*)malloc(sizeof(*p));
	if(p==NULL)
	{
		printf("No hay memoria\n");
		return 1;
	}
	else
	{
		printf("Comienze a ingresar los datos (0 para terminar) \n");

			base=p;

			printf("Ingrese un entero: ");
			n=getint();

			(*p).dato=n;
			(*p).sig=NULL;
			printf("Ingrese un entero :");

			while( (n=getint())!=0)
			{
				q=(struct nodo*)malloc(sizeof(*q));
				if(q==NULL)
				{
					printf("no hay memoria\n");
					return 1;
				}
				else
				{
						(*p).sig=q;
						(*q).dato=n;
						(*q).sig=NULL;
						p=q;
					printf("Ingrese un entero: ");
				}
			}
	}
```

```
	printf("\nLos datos ingresados son:\n");
		imprime(base);

	free(base);	free(p);		free(q);

	system("PAUSE");
	return 0;
}

int getint(void)
{
	int n;
		scanf("%d", &n);
	return n;
}

void imprime(struct nodo *p)
{
	while(p!=NULL)
	{
		printf("%d \t", (*p).dato);
		p=(*p).sig;
	}
}
```

2) Codificar un programa que permita cargar una cola dinámica con N letras (cualquier carácter que no sea letra rechazarlo, y pedir un nuevo ingreso) ingresados desde el teclado. Recorrerla y mostrar su contenido por pantalla.

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	char dato;
	struct nodo *sig;
};

char getletra(void);
void imprime(struct nodo *);

int main()
{
	struct nodo *p, *q, *base;
	char c;

	p=NULL;
	q=NULL;
	base=NULL;

	printf("Ingrese caracteres ('s' para terminar)\n");

	p=(struct nodo*)malloc(sizeof(*p));
	if(p!=NULL)
	{
		printf("---Nuevo Nodo---\n");
		if( ((*p).dato=getletra()) !='s' )
		{
				base=p;
				(*p).sig=NULL;
```

```
                                        printf("---Nuevo Nodo---\n");
                                        while( (c=getletra()) !='s' )
                                        {
                                                q=(struct nodo*)malloc(sizeof(*q));
                                                if(q!=NULL)
                                                {
                                                        (*q).dato=c;
                                                        (*q).sig=NULL;
                                                        (*p).sig=q;
                                                        p=q;
                                                }
                                                else
                                                {
                                                        printf("No hay memoria\n");
                                                        return 1;
                                                }
                                                printf("---Nuevo Nodo---\n");
                                        }
                                system("CLS");
                                printf("Los datos ingresados son: \n");
                                imprime(base);
                        }
                        else
                        {
                                system("CLS");
                                printf("La estructura quedo vacia\n");
                        }

        }
        else
        {
                printf("No hay memoria \n");
                return 1;
        }

        system("PAUSE");
        return 0;
}

char getletra(void)
{
        char c[2];

                do
                {
                        printf("Ingrese dato: ");
                        scanf("%s", c);
                }while( (c[0]>'Z' && c[0]<'a') || c[0]>'z'  || c[0]<'A');

        return c[0];
}

void imprime(struct nodo *p)
{
        while(p!=NULL)
        {
                printf("%c\n", (*p).dato);
                p=(*p).sig;
        }
}
```

3) Se desea codificar una Cola dinámica. Los nodos contienen Apellido y edad de personas. Cargar la estructura hasta que se ingrese la edad 999. Se desea recorrerla y mostrar por pantalla la información de los nodos que representen personas mayores de edad (21 años).

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
	int edad;
	char apellido[30];
	struct nodo *sig;
};

int getint(void);

int main()
{
	struct nodo *p, *q, *base;
	int n, i=1;

	p=NULL;
	q=NULL;
	base=NULL;

	printf("Ingrese datos ( edad=999 para terminar)\n\n");

	printf("\n------- Datos ------\n");
	printf("Ingrese edad: ");
	n=getint();
	if(n!=999)
	{
		p=(struct nodo*)malloc(sizeof(*p));
		if(p!=NULL)
		{
			base=p;
			printf("Ingrese apellido: ");
			scanf("%s", (*p).apellido);
			(*p).edad=n;
			(*p).sig=NULL;

			printf("\n------- Datos ------\n");
			printf("Ingrese edad: ");
			while( (n=getint())!= 999 )
			{
				q=(struct nodo*)malloc(sizeof(*q));
				if(q!=NULL)
				{
					(*q).edad=n;
					printf("Ingrese apellido: ");
					scanf("%s", (*q).apellido);
					(*q).sig=NULL;
					(*p).sig=q;
					p=q;
				}
				else
				{
					printf("No hay memoria\n");
					return 1;
				}
```

```
                        printf("\n------- Datos ------\n");
                                printf("Ingrese edad: ");
                        }
                }
                else
                {
                                printf("No hay memoria\n");
                                return 1;
                }
        }
        else
        {
                printf("\nNo se ingreso ningun dato\n");
        }

        p=base;
        while( p!=NULL)
        {
                if( (*p).edad>=21 )
                {
                        printf("Aplellido: %s\n", (*p).apellido);
                        printf("Edad: %d\n", (*p).edad);
                }
                p=(*p).sig;
        }

        system("PAUSE");
                return 0;
}

int getint(void)
{
        int n;
                scanf("%d", &n);
        return n;
}
```

4) Se desea codificar una cola dinámica. Los nodos contienen Apellido, edad y sexo de personas. Cargar la estructura hasta que se ingrese la edad 999. Se desea recorrerla y generar una pila dinámica que solo contenga los nodos pertenecientes a personas que pueden votar del sexo femenino. Mostrar por pantalla.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

struct nodo
{
        char sexo[1];
        int edad;
        char apellido[30];
        struct nodo *sig;
};

int getint(void);

int main()
{
        struct nodo *p, *q, *base, *base2;
        int n;
```

```
//ingreso de datos
p=(struct nodo*)malloc(sizeof(*p));
if(p!=NULL)
{
	base=p;
	printf("\n------------------------\n");
	printf("Ingrese edad: ");
	if ( (n=getint()) != 999)
	{
		(*p).edad=n;
		printf("Ingrese apellido: ");
		scanf("%s", (*p).apellido);
		printf("Ingrese sexo: ");
		scanf("%s", (*p).sexo);
		(*p).sig=NULL;

		printf("\n------------------------\n");
		printf("Ingrese edad: ");
		while( (n=getint())!= 999)
		{
			q=(struct nodo*)malloc(sizeof(*q));
			if( q!=NULL)
			{
				(*q).edad=n;
				printf("Ingrese apellido: ");
				scanf("%s", (*q).apellido);
				printf("Ingrese sexo: ");
				scanf("%s", (*q).sexo);
				(*q).sig=NULL;
				(*p).sig=q;
				p=q;
			}
			else
			{
				printf("No hay memoria\n");
				return 1;
			}
		printf("\n------------------------\n");
		printf("Ingrese edad: ");
		}
	}
}
else
{
	printf("No hay memoria\n");
	return 1;
}

p=base;
base2=NULL;
while( p!=NULL )
{
	if( ((*p).edad>=18) && ((*p).sexo[0]=='F') )
	{
		q=(struct nodo*)malloc(sizeof(*q));
		if(q!=NULL)
		{
			(*q).edad=(*p).edad;
			strcpy( (*q).apellido, (*p).apellido );
			(*q).sig=base2;
			base2=q;
		}
```

```
                        else
                        {
                                printf("No hay memoria\n");
                                return 1;
                        }
                }
                p=(*p).sig;
        }

        printf("\n----Mujeres que pueden votar-----\n");
        while( q!=NULL)
        {
                printf("Apellido: %s \n", (*q).apellido);
                q=(*q).sig;
        }

        free(p);
        free(q);

        system("PAUSE");
        return 0;
}

int getint(void)
{
        int n;
                scanf("%d", &n);
        return n;
}
```

5) Se desea codificar una cola dinámica. Los nodos contienen números enteros. Se deberá cargarla hasta que el usuario decida dejar de hacerlo, mediante ingreso por teclado. Recorrerla y mostrar por pantalla la primera mitad del contenido de la estructura.

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
        int dato;
        struct nodo *sig;
};

int getint(void);

int main()
{
        struct nodo *p, *q, *base;
        int n, i, cont;

        base=NULL;
        q=NULL;
        p=NULL;

        p=(struct nodo*)malloc(sizeof(*p));
        if(p!=NULL)
        {
                base=p;
                printf("Ingrese un entero: ");
                n=getint();
```

```
                (*p).dato=n;
                (*p).sig=NULL;
                printf("Ingrese un entero: ");
                while( (n=getint())!= 0)
                {
                        q=(struct nodo*)malloc(sizeof(*q));
                        if(q!=NULL)
                        {
                                (*q).dato=n;
                                (*q).sig=NULL;
                                (*p).sig=q;
                                p=q;
                                printf("Ingrese un entero: ");
                        }
                        else
                        {
                                printf("No hay memoria\n");
                                return 1;
                        }
                }
        }
        else
        {
                printf("No hay memoria\n");
                return 1;
        }

        cont=0;
        p=base;
        while(p!=NULL)
        {
                cont++;
                p=(*p).sig;
        }

        p=base;
        system("CLS");
        printf("La primera mitad de la estructura tipo cola es: \n");
        for(i=0 ;i<cont/2 ;i++)
        {
                printf("%d\n", (*p).dato);
                p=(*p).sig;
        }

        system("PAUSE");
        return 0;
}

int getint(void)
{
        int n;
                scanf("%d", &n);
        return n;
}
```

6) Leer y almacenar en un arreglo estático los nombres de N alumnos, junto con su promedio general y un entero que represente el número de inasistencias a clase.
Generar una cola dinámica con la información del arreglo. Recorrerla y mostrar mediante un menú, las siguientes opciones:

a) la información del nodo que contiene el promedio mayor

b) la información del nodo que tiene menor número de inasistencias.

c) Los nombres de los alumnos cuyo promedio sea mayor a 7.00.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define tam 3

struct nodo
{
	char nombre[30];
	char apellido[30];
	int faltas;
	float promedio;
	struct nodo *sig;
};

struct alumno
{
	char nombre[30];
	char apellido[30];
	int faltas;
	float promedio;
}ficha[tam];

int main()
{
	struct nodo *p, *base, *q, *base2, *r;
	int  opcion, i, min, cont;
	float max;

	do
	{
		printf("\n\n----MENU----\n");
		printf("1. Ingresar datos \n");
		printf("2. Alumnos con mayor promedio\n");
		printf("3. Alumnos con menos inasistencias \n");
		printf("4. Alumnos con promedio mayor a 7 \n");
		printf("5. Salir \n");
		printf("Ingrese la opcion elegida: ");
		scanf("%d", &opcion);

		switch(opcion)
		{
			case 1:
				printf("\n------------Ingreso de datos------------\n");
				for(i=0 ;i<tam ;i++)
				{
					printf("\n---- ALUMNO %d ----\n", i+1);
					printf("Nombre: ");
					scanf("%s",  ficha[i].nombre);
					printf("Apellido: ");
					scanf("%s", ficha[i].apellido);
					printf("Inasistencias: ");
```

```
                scanf("%d", &ficha[i].faltas);
                printf("Promedio: ");
                scanf("%f", &ficha[i].promedio);
        }

        p=NULL;
        base=NULL;
        cont=0;
                                //Copia elementos del arreglo en una cola
        for(i=0 ;i<tam ;i++)
        {
                if(cont==0)
                {
                        p=(struct nodo*)malloc(sizeof(*p));
                        if(p==NULL)
                        {
                                printf("No se pudo gestionar la memoria\n");
                                return 1;
                        }
                        else
                        {
                                base=p;
                                strcpy((*p).nombre, &ficha[i].nombre[0]);
                                strcpy((*p).apellido, &ficha[i].apellido[0]);
                                (*p).faltas=ficha[i].faltas;
                                (*p).promedio=ficha[i].promedio;
                                (*p).sig=NULL;
                        }
                }
                else
                {
                        q=(struct nodo*)malloc(sizeof(*q));
                        if(q==NULL)
                        {
                                printf("No se pudo gestionar la memoria\n");
                                return 1;
                        }
                        else
                        {
                                strcpy((*q).nombre, &ficha[i].nombre[0]);
                                strcpy((*q).apellido, &ficha[i].apellido[0]);
                                (*q).faltas=ficha[i].faltas;
                                (*q).promedio=ficha[i].promedio;
                                (*q).sig=NULL;
                                (*p).sig=q;
                                p=q;
                        }
                }
                cont=1;
        }
    break;
case 2:
    p=base;
    max=0;
    cont=0;
        while(p!=NULL)
        {
                if( (*p).promedio>=max )
                {
                        if(cont==0)
                        {
                                q=(struct nodo*)malloc(sizeof(*q));
```

```
                                        if(q==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
                                        {
                                                base2=q;
                                                max=(*p).promedio;
                                                (*q).promedio=(*p).promedio;
                                                strcpy( (*q).nombre, (*p).nombre );
                                                strcpy( (*q).apellido, (*p).apellido );
                                                (*q).sig=NULL;
                                        }
                                }
                                else
                                {
                                        r=(struct nodo*)malloc(sizeof(*r));
                                        if(r==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
                                        {
                                                max=(*p).promedio;
                                                (*r).promedio=(*p).promedio;
                                                strcpy( (*r).nombre, (*p).nombre );
                                                strcpy( (*r).apellido, (*p).apellido );
                                                (*r).sig=NULL;
                                                (*q).sig=r;
                                                q=r;
                                        }
                                }
                        }
                        cont=1;
                        p=(*p).sig;
                }
                printf("\nEl mayor promedio es: %.2f: \n", max);
                q=base2;
                        while(q!=base2)
                {
                        printf("Del alumno %s %s\n", (*q).nombre, (*q).apellido);
                        base=(*q).sig;
                }
        break;
case 3:
        p=base;
        min=999;
                while(p!=NULL)
                {
                        if( (*p).faltas<=min )
                        {
                                if(cont==0)
                                {
                                        q=(struct nodo*)malloc(sizeof(*q));
                                        if(q==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
```

```
                                        {
                                                base2=q;
                                                min=(*p).faltas;
                                                (*q).faltas=(*p).faltas;
                                                strcpy( (*q).nombre, (*p).nombre );
                                                strcpy( (*q).apellido, (*p).apellido );
                                                (*q).sig=NULL;
                                        }
                                }
                                else
                                {
                                        r=(struct nodo*)malloc(sizeof(*r));
                                        if(r==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
                                        {
                                                min=(*p).faltas;
                                                (*r).faltas=(*p).faltas;
                                                strcpy( (*r).nombre, (*p).nombre );
                                                strcpy( (*r).apellido, (*p).apellido );
                                                (*r).sig=NULL;
                                                (*q).sig=r;
                                                q=r;
                                        }
                                }
                        cont=1;
                        p=(*p).sig;
                }
                printf("\nLos alunos con menos inasistencias son: \n");
                q=base2;
                        while(q!=NULL)
                {
                        printf("%s %s\n", (*q).nombre, (*q).apellido);
                        q=(*q).sig;
                }
                printf("Con %d inasistencias\n", min);
        break;
case 4:
        p=base;
        max=7;
        base2=NULL;
        cont=0;
                while(p!=NULL)
                {
                        if( (*p).promedio>=max )
                        {
                                if(cont==0)
                                {
                                        q=(struct nodo*)malloc(sizeof(*q));
                                        if(q==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
                                        {
                                                max=(*p).promedio;
                                                (*q).promedio=(*p).promedio;
                                                strcpy( (*q).nombre, (*p).nombre );
```

```
                                                strcpy( (*q).apellido, (*p).apellido );
                                                (*q).sig=NULL;
                                                base2=q;
                                        }
                                }
                                else
                                {
                                        r=(struct nodo*)malloc(sizeof(*q));
                                        if(r==NULL)
                                        {
                                                printf("No se pudo gestionar la memoria \n");
                                                return 1;
                                        }
                                        else
                                        {
                                                max=(*p).promedio;
                                                (*r).promedio=(*p).promedio;
                                                strcpy( (*r).nombre, (*p).nombre );
                                                strcpy( (*r).apellido, (*p).apellido );
                                                (*r).sig=NULL;
                                                (*q).sig=r;
                                                q=r;
                                        }
                                }
                        }
                        cont=1;
                        p=(*p).sig;
                }
                printf("\nLos alumnos con promedio mayor a 7 son: \n");
                q=base2;
                        while(q!=NULL)
                {
                        printf("%s %s ", (*q).nombre, (*q).apellido);
                        printf("\tPromedio: %.2f\n", (*q).promedio);
                        q=(*q).sig;
                }
                free(q);
                break;
        case 5:
                        printf("\n\n----FIN DEL PROGRAMA----\n\n");
                break;
        default:
                        system("CLS");
                printf("Ingrese una opcion valida\n\n");
                break;
        }
}while(opcion!=5);

system("PAUSE");
return 0;
}
```

7) Generar una estructura dinámica tipo cola la que permita guardar en un nodo la información correspondiente a temperatura y presión de una máquina. Generar un menú con las opciones:

- ***a) agregar un nodo nuevo.***
- ***b) borrar un nodo***
- ***c) recorrerla y mostrar su contenido en pantalla.***
- ***d) mostrar por pantalla la máxima y la mínima temperatura, con la posición que ocupan en la estructura***
- ***e) mostrar por pantalla la máxima y la mínima presión, con la posición que ocupan en la estructura***

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
	float temp;
	float pres;
	struct nodo *sig;
};

void imprime(struct nodo *);

void getlectura(struct nodo *);

struct nodo* put1(void);

struct nodo* put2(struct nodo *);

int main()
{
	struct nodo *p, *q, *base;
	int opcion, cont=0, i, j, pos1, pos2;
	float max, min;

	p=NULL;
	q=NULL;
	base=NULL;

	do
	{
		opcion=menu();
		switch(opcion)
		{
			case 1:
				if(cont==0)
				{
					base=put1();
					p=base;
				}
				else
				{
					p=put2(p);
				}
				cont=1;
				break;
			case 2:
				if(base==NULL)
				{
					printf("La estructura esta vacia\n");
				}
				else
```

```
	{
		q=(*base).sig;
		free(base);
		base=q;
	}
	break;
case 3:
	system("CLS");
		printf("Las lecturas son: \n");
		q=base;
		imprime(q);
	break;
case 4:
	q=base;
	max=(*q).temp-1;
	min=(*q).temp+1;
	i=1;
	j=1;
		while(q!=NULL)
		{
			if( (*q).temp>max )
			{
				pos1=i;
				max=(*q).temp;
			}
		 q=(*q).sig;
		 i++;
		}
	 q=base;
	 while(q!=NULL)
		{
			if( (*q).temp<min )
			{

				pos2=j;
				min=(*q).temp;
			}
		 q=(*q).sig;
		 j++;
		}
	  system("CLS");
	  printf("La temperatura maxima es: %.2f  ",max );
	  printf("de la lectura %d\n\n", pos1);
	  printf("La temperatura minima es: %.2f  ",min );
	  printf("de la lectura %d\n\n", pos2);
	break;
case 5:
	q=base;
	max=(*q).pres-1;
	min=(*q).pres+1;
	i=1;
	j=1;
		while(q!=NULL)
		{
			if( (*q).pres>max )
			{
				pos1=i;
				max=(*q).pres;
			}
		 q=(*q).sig;
		 i++;
		}
```

```
                                        q=base;
                                                while(q!=NULL)
                                                {
                                                        if( (*q).pres<min )
                                                        {
                                                                pos2=j;
                                                                min=(*q).pres;
                                                        }
                                                 q=(*q).sig;
                                                 j++;
                                                }
                                                system("CLS");
                                         printf("La presion maxima es: %.2f  ",max );
                                         printf("de la lectura %d\n\n", pos1);
                                         printf("La presion minima es: %.2f  ",min );
                                         printf("de la lectura %d\n\n", pos2);
                                        break;
                                case 6:
                                        system("CLS");
                                                printf("Fin del programa\n\n\n");
                                        break;
                                default:
                                                printf("\n\nERROR! Ingrese una opcion valida\n\n");
                                        break;
                        }
        }while(opcion!=6);


        system("PAUSE");
        return 0;
}


int menu(void)
{
        int opc;
                printf("\n---- MENU ----\n");
                printf("1. Insertar nodo\n");
                printf("2. Borrar nodo \n");
                printf("3. Mostrar por pantalla\n");
                printf("4. Maxima y minima temperatura\n");
                printf("5. Maxima y minima presion\n");
                printf("6. Finalizar el programa\n");
                printf("Ingrese opcion elegida: ");
                scanf("%d", &opc);
        return opc;
}

void getlectura(struct nodo *p)
{
                printf("\n--- Nuevo Nodo ---\n");
                printf("Ingrese temperatura: ");
                scanf("%f", &(*p).temp);
                printf("Ingrese presion: ");
                scanf("%f", &(*p).pres);
                printf("\n\n");
}
```

```
void imprime(struct nodo *p)
{
	int i=1;
	while(p!=NULL)
	{
		printf("\n---- Lectura N:%d ----\n", i);
		printf("Temperatura: %.2f\n", (*p).temp);
		printf("Presion: %.2f\n", (*p).pres);
		i++;
		p=(*p).sig;
	}
}

struct nodo* put1(void)
{
	struct nodo *p;

		p=(struct nodo*)malloc(sizeof(*p));
		if(p!=NULL)
		{
			getlectura(p);
			(*p).sig=NULL;
			return p;
		}
		else
		{
			printf("No hay memoria\n");
		}
}

struct nodo* put2(struct nodo *p)
{
	struct nodo *q;

		q=(struct nodo*)malloc(sizeof(*q));
		if(q!=NULL)
		{
			getlectura(q);
			(*q).sig=NULL;
			(*p).sig=q;
			return q;
		}
		else
		{
			printf("No hay memoria\n");
		}
}
```

8) Se tienen cargadas dos estructuras de tipo cola. La información de los nodos de cada estructura, son números enteros sin repetición, Cada estructura contiene información ordenada en forma creciente, y no necesariamente tienen los mismos elementos ni el mismo número de nodos. Generar otra estructura del mismo tipo que contenga los datos de las dos primeras. La cola de salida deberá quedar ordenada de forma creciente, en el mismo momento de generación. Recorrerla y mostrar la intercalación.

```
#include <stdio.h>
#include <stdlib.h>

struct nodo
{
	int dato;
	struct nodo *sig;
};

int getint(void);

void imprime(struct nodo *);

void ordena(struct nodo *);

struct nodo *put1(int );

struct nodo *put2(struct nodo *, int );

struct nodo *ingresa(struct nodo *);

struct nodo *enlaza(struct nodo *, struct nodo *);

int main()
{
	struct nodo *p=NULL, *q=NULL, *base=NULL, *base2=NULL, *base3=NULL;

	printf("\nIngrese la primera estructura\n\n");
	base=ingresa(base);

	printf("\nIngrese la segunda estrutura\n\n");
	base2=ingresa(base2);

	system("CLS");
	printf("La estructura 1 es: \n");
	p=base;
	ordena(p);
	p=base;
	imprime(p);

	printf("\n\nLa estructura 2 es: \n");
	p=base2;
	ordena(p);
	p=base2;
	imprime(p);

	printf("\n\nLa estructura mezclada es: \n");
	p=base;
	q=base2;
	base3=enlaza(p, q);
	imprime(base3);

	system("PAUSE");
	return 0;
```

```
}

int getint(void)
{
	int n;
		printf("--- Nuevo Nodo ---\n");
		printf("Ingrese entero: ");
		scanf("%d", &n);
		printf("\n\n");
	return n;
}

void imprime(struct nodo *p)
{
	while(p!=NULL)
	{
		printf("%d\n", (*p).dato);
		p=(*p).sig;
	}
}

void ordena(struct nodo *p)
{
	struct nodo *q;
	int aux;

	while(p!=NULL)
	{
		q=(*p).sig;
		while(q!=NULL)
		{
			if( (*p).dato > (*q).dato)
			{
				aux=(*p).dato;
				(*p).dato=(*q).dato;
				(*q).dato=aux;
			}
			q=(*q).sig;
		}
		p=(*p).sig;
	}
}


struct nodo *ingresa(struct nodo *base)
{
	struct nodo *p, *q;
	int n;

	p=(struct nodo*)malloc(sizeof(*p));
	if(p!=NULL)
	{
		if( (n=getint()) != 0)
		{
			base=p;
			(*p).dato=n;
			(*p).sig=NULL;
			while( (n=getint()) != 0)
			{
```

```
                                        q=(struct nodo*)malloc(sizeof(*q));
                                        if(q!=NULL)
                                        {
                                                (*q).dato=n;
                                                (*q).sig=NULL;
                                                (*p).sig=q;
                                                p=q;
                                        }
                                        else
                                        {
                                                printf("No hay memoria\n");
                                        }
                                }
                        }
                        else
                        {
                                printf("\nLa lista quedo vacia\n");
                        }
        }
        else
        {
                printf("No hay memoria\n");
        }
        return base;
}

struct nodo *put1(int n)
{
        struct nodo *p, *base;

        p=(struct nodo*)malloc(sizeof(*p));
        if(p!=NULL)
        {
                base=p;
                (*p).dato=n;
                (*p).sig=NULL;
        }
        else
        {
                printf("No hay memoria\n");
        }
        return p;
}

struct nodo *put2(struct nodo *p, int n)
{
        struct nodo *q;

        q=(struct nodo*)malloc(sizeof(*q));
        if(q!=NULL)
        {
                (*q).dato=n;
                (*q).sig=NULL;
                (*p).sig=q;
                return q;
        }
        else
        {
                printf("No hay memoria\n");
        }
}
```

```
struct nodo *enlaza(struct nodo *p, struct nodo *q)
{
	struct nodo *p2, *base;
	int n, cont=0;

	while( p!=NULL || q!=NULL )
	{
		if( p!=NULL && q!=NULL)
		{
			if( (*p).dato <= (*q).dato )
			{
				n=(*p).dato;
				if(cont==0)
				{
					base=put1(n);
					p2=base;
				}
				else
				{
					p2=put2(p2,n);
				}
				p=(*p).sig;
			}
			else
			{
				n=(*q).dato;
				if(cont==0)
				{
					base=put1(n);
					p2=base;
				}
				else
				{
					p2=put2(p2,n);
				}
				q=(*q).sig;
			}
		}
		else
		{
			if(p!=NULL)
			{
				n=(*p).dato;
				if(cont==0)
				{
					base=put1(n);
					p2=base;
				}
				else
				{
					p2=put2(p2,n);
				}
				p=(*p).sig;
			}
			else
			{
				n=(*q).dato;
				if(cont==0)
				{
					base=put1(n);
					p2=base;
				}
```

```
                                else
                                {
                                        p2=put2(p2,n);
                                }
                                p=(*p).sig;
                        }
                }
                cont=1;
        }
        return base;
}
```

9) Se tienen cargada una estructura de tipo cola. La información de los nodos son números enteros, la cola admite tener nodos con datos repetidos. Generar otra estructura del mismo tipo sin las repeticiones. Recorrer las dos estructuras dinámicas y mostrar por pantalla sus contenidos.

```
#include<stdio.h>
#include<stdlib.h>

struct nodo
{
        int dato;
        struct nodo *sig;
};

int getint(void);
void imprime(struct nodo *);

int main()
{
        struct nodo *p, *q, *base, *p2, *q2, *base2;
        int n, flag, cont;

        //ingreso la primera cola
        p=(struct nodo*)malloc(sizeof(*p));
        if(p!=NULL)
        {
                if( (n=getint()) != 0)
                {
                        base=p;
                        (*p).dato=n;
                        (*p).sig=NULL;
                        while( (n=getint()) != 0)
                        {
                                q=(struct nodo*)malloc(sizeof(*q));
                                if(q!=NULL)
                                {
                                        (*q).dato=n;
                                        (*q).sig=NULL;
                                        (*p).sig=q;
                                        p=q;
                                }
                                else
                                {
                                        printf("No hay memoria\n");
                                        return 1;
                                }
                        }
                }
```

```
		else
		{
			printf("\nLa lista quedo vacia\n");
		}
}
else
{
		printf("No hay memoria\n");
		return 1;
}

//creo cola sin elementos repetidos
p=base;
flag=0;
cont=0;
while(p!=NULL)
{
		q=(*p).sig;
		while( q!=NULL )
		{
			if( (*p).dato==(*q).dato)
			{
				flag=1;
			}
			q=(*q).sig;
		}
		if(flag!=1)
		{
			if(cont==0)
			{
				p2=(struct nodo*)malloc(sizeof(*p2));
				if(p2!=NULL)
				{
					base2=p2;
					(*p2).dato=(*p).dato;
					(*p2).sig=NULL;
				}
				else
				{
					printf("No hay memoria\n");
					return 1;
				}
			}
			else
			{
				q2=(struct nodo*)malloc(sizeof(*q2));
				if(q2!=NULL)
				{
					(*q2).dato=(*p).dato;
					(*q2).sig=NULL;
					(*p2).sig=q2;
					p2=q2;
				}
				else
				{
					printf("No hay memoria\n");
					return 1;
				}
			}
			cont++;
		}
		p=(*p).sig;
```

```
                flag=0;
        }

        //imprimo la primera cola ingresada
        system("CLS");
        printf("La estructura original es: \n");
        imprime(base);

        printf("\n\nLa estructura sin elementos iguales es: \n");
        imprime(base2);


        system("PAUSE");
        return 0;
}

int getint(void)
{
        int n;
                printf("--- Nuevo Nodo ---\n");
                printf("Ingrese entero: ");
                scanf("%d", &n);
                printf("\n\n");
        return n;
}

void imprime(struct nodo *p)
{
        while(p!=NULL)
        {
                printf("%d\n", (*p).dato);
                p=(*p).sig;
        }
}
```